

ACCRA TECHNICAL UNIVERSITY, GHANA

FACULTY OF APPLIED SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

BCP 210: Data Structures and Algorithms I

Coursework Assignment 1

Academic Year: 2025/2026

Student Name: Your Name: TAWIAH THEOPHILUS NOMO

Index Number: 01242947B

Assignment Report

Introduction

This assignment focuses on the implementation of fundamental data structures and algorithms using Python. The objective is to develop efficient solutions for solving practical problems involving arrays and singly linked lists while considering algorithm correctness, efficiency, and edge-case handling. The solutions were implemented using Python 3 and tested under different scenarios to ensure reliability

Assignment 1: Arrays – University Course Registration Analytics

Objective

The objective of this assignment was to analyze a university course registration list using efficient array algorithms.

Task 1: Most Frequent Registrant

A dictionary (hash map) was used to count the frequency of every student ID in the registration list. During traversal, the algorithm continuously tracked the student with the highest frequency.

Worst-case Time Complexity: $O(n)$

Worst-case Space Complexity: $O(n)$

Task 2: Ordered Deduplication

A set was used to keep track of student IDs that had already been encountered. Each new unique student ID was added to the result list while preserving its first appearance.

Worst-case Time Complexity: $O(n)$

Worst-case Space Complexity: $O(n)$

Task 3: First Unique Student

The algorithm first counted the occurrences of every student ID using a dictionary. It then traversed the list again to locate the first student ID whose frequency was exactly one.

Worst-case Time Complexity: $O(n)$

Worst-case Space Complexity: $O(n)$

Task 4: Contiguous Subarray Sum

The prefix sum technique was used together with a hash set to determine whether any contiguous sequence of student IDs had a sum equal to a specified target value. This approach avoids checking every possible subarray and significantly improves efficiency.

Worst-case Time Complexity: $O(n)$

Worst-case Space Complexity: $O(n)$

Expected Output

Most frequent student: 1023

Unique registrations: [1023, 1050, 1102, 1201, 1300]

First non-repeated student: 1201

Subarray with target sum exists: True

Assignment 2: Linked Lists – University Helpdesk Ticket Queue

Objective

The objective of this assignment was to implement a Singly Linked List from scratch to simulate a university helpdesk ticket queue.

Task 1: Enqueue Ticket

A new ticket node was inserted at the end of the linked list using a tail pointer.

Worst-case Time Complexity: $O(1)$

Task 2: Priority Insert

The linked list was traversed until the specified ticket ID was found. The new priority ticket was then inserted immediately after it.

Worst-case Time Complexity: $O(n)$

Task 3: Resolve Ticket

The linked list was searched for the specified ticket ID, after which the corresponding node was removed while maintaining the integrity of the list.

Worst-case Time Complexity: $O(n)$

Task 4: Find Middle Ticket

The fast and slow pointer technique was used. The fast pointer moved two nodes at a time while the slow pointer moved one node. When the fast pointer reached the end of the list, the slow pointer pointed to the middle node.

Worst-case Time Complexity: $O(n)$

Task 5: Reverse Queue

The linked list was reversed by changing node pointers without copying any data.

Worst-case Time Complexity: $O(n)$

Task 6: Display Queue

The linked list was traversed from the head node to the tail node while displaying every ticket.

Worst-case Time Complexity: $O(n)$

Complexity Summary

Operation	Time Complexity	Space Complexity
Enqueue Ticket	$O(1)$	$O(1)$
Priority Insert	$O(n)$	$O(1)$
Resolve Ticket	$O(n)$	$O(1)$
Find Middle Ticket	$O(n)$	$O(1)$
Reverse Queue	$O(n)$	$O(1)$
Display Queue	$O(n)$	$O(1)$

Testing

The implemented programs were tested using both normal and boundary cases.

The following scenarios were verified:

- Empty lists
- Single-element lists
- Duplicate values
- Unique values
- Missing ticket IDs
- Queue reversal
- Priority insertion
- Ticket deletion
- Middle ticket identification

- Target subarray sum existence

All test cases executed successfully without runtime errors.

Bonus Challenge

Question

The university IT department currently stores tickets in a singly linked list. If the helpdesk workload shifts such that they perform thousands of searches per day but only a few insertions and deletions, would a Python list (contiguous array) be more appropriate? Justify your answer thoroughly.

Answer

Yes. If the workload consists primarily of thousands of search operations with only occasional insertions and deletions, a Python list would generally be a more appropriate choice than a singly linked list.

Although both a Python list and a singly linked list require $O(n)$ time for a linear search, Python lists are usually much faster in practice because of CPU cache locality. The elements of a Python list are stored in contiguous memory locations, allowing the processor to load several nearby elements into the CPU cache at once. This significantly reduces memory access time during sequential searches.

In contrast, the nodes of a singly linked list are allocated separately in memory. Each search requires following pointers from one node to the next, a process known as pointer chasing. Since the nodes are scattered throughout memory, cache misses occur more frequently, making searches slower despite having the same Big-O time complexity.

Memory allocation also differs between the two structures. A Python list stores only the elements themselves in contiguous memory, making it more memory efficient. A linked list stores both the data and an additional pointer for every node, increasing memory usage and allocation overhead.

Although linked lists provide efficient insertions and deletions after a node has been located, these advantages become less important when modifications occur infrequently. Since the primary workload in this scenario is searching rather than updating the data structure, the faster memory access and better cache performance of Python lists outweigh the insertion and deletion benefits of linked lists.

Therefore, for a workload dominated by searches with relatively few insertions and deletions, a Python list is the more suitable data structure because it offers better practical performance, improved cache efficiency, lower memory overhead, and simpler implementation.

Conclusion

This assignment provided practical experience in implementing fundamental data structures and algorithms using Python. Efficient solutions were developed for array-based problems and linked list operations while carefully analyzing their computational complexity. Appropriate testing was performed to ensure correctness and reliability. The bonus discussion further demonstrated the importance of selecting data structures based not only on theoretical Big-O complexity but also on practical hardware considerations such as CPU cache locality and memory allocation.